Fill in group number and member names:

In [113…

```
GROUP = "7"
NAME1 = "Jakob Söderberg"
NAME2 = "Jonathan Möller"
```

Optimization for learning - FRTN50

Assignment 1

The goal of this assignment is to become familiar with some of the steps involved in solving an optimization problem. In this assignment, you will form Fenchel dual problems, find gradients and/or proximal operators, and implement the proximal gradient method.

Problem The problem we will solve is the following constrained problem

$$\underset{x \in S}{\text{minimize}} \ \tfrac{1}{2} x^T Q x + q^T x \tag{1}$$

where $Q \in \mathbb{S}^n_{++}$, $q \in \mathbb{R}^n$ and $S \subseteq \mathbb{R}^n$ is a set defined by the points $a, b \in \mathbb{R}^n$, $a \leq b$, such that

$$S = \{x \in \mathbb{R}^n : a \leq x \leq b\}.$$

I.e., we are going to minimize a quadratic function over an n-dimensional box. Recall that, the vector inequality $a \leq b$ means that

$$a_i \leq b_i$$

for each $i = 1, \ldots, n$. Define the function $f : \mathbb{R}^n \to \mathbb{R}$ such that

$$f(x) = \tfrac{1}{2} x^T Q x + q^T x$$

for each $x \in \mathbb{R}^n$ and let $\iota_S : \mathbb{R}^n \to \mathbb{R} \cup \{\infty\}$ denote the indicator function of the set S, i.e.,

$$\iota_S(x) = \begin{cases} 0 & \text{if } x \in S, \\ \infty & \text{if } x \in \mathbb{R}^n \setminus S. \end{cases}$$

Problem (1) can then be written as

$$\underset{x \in \mathbb{R}^n}{\text{minimize}} \ f(x) + \iota_S(x). \tag{2}$$

Solution method To solve optimization problem (2), we will use the *proximal gradient method*. It solves problems of the form

$$\underset{x \in \mathbb{R}^n}{\text{minimize}} \ f(x) + g(x) \tag{3}$$

where $f : \mathbb{R}^n \to \mathbb{R}$ is differentiable and $g : \mathbb{R}^n \to \mathbb{R} \cup \{\infty\}$ is proximable, i.e., $\mathrm{prox}_{\gamma g}$ can be cheaply computed. The proximal gradient method (with constant step-size) is given by:

- Pick some arbitrary initial guess $x^0 \in \mathbb{R}^n$ and step-size $\gamma > 0$.
- For $k = 0, 1, 2 \ldots$, let

$$x^{k+1} = \mathrm{prox}_{\gamma g}\big(x^k - \gamma \nabla f(x^k)\big). \tag{4}$$

- Stop when x^k is deemed to have converged.

In this assignment, we simply run the proximal gradient method a large fixed number of iterations and plot the norm of the fixed-point residual $||x^{k+1} - x^k||_2$ (also known as the step-length), of each step to make sure it converges to zero. Since the experiments are run on a computer, zero means smaller than machine precision, which usually is around 10^{-15}.

The step-size parameter γ in the (4) will affect the convergence. It should be tuned to the problem or chosen based on properties of f and g. In particular, suppose that f and g are proper, closed and convex. If f is β-smooth for some parameter $\beta > 0$, the maximal step-size to guarantee convergence is $\gamma < \frac{2}{\beta}$.

Below are the tasks that you need to solve. Keep this in mind:

- The suggested exercises in the exercise compendium found on the Canvas course page, up until and including the chapter "Proximal gradient method - basics", is relevant for this assignment.
- Carefully motivate every step in your calculations.
- Use **figures** and **tables** to motivate your answers.
- Figures must have appropriately labeled axes and must be referenced in the main text.
- Your code should be written in a quite general manner, i.e., if a question is slightly modified, it should only require slight modifications in your code as well.
- Comment your code well.
- Make sure you plot in such a way that small quantities (e.g., $||x^{k+1} - x^k||_2$) are visible. In particular, use log-linear plots, where the quantity that should go to 0 is on the y-axis using logarithmic scale, and the iteration number k on the x-axis using linear scale.
- What you need to submit to Canvas:
  - This jupyter notebook containing your solutions.
  - An exported pdf version of the jupyter notebook. (One way to do this is to print the notebook in your web browser, and then save as pdf.)

This table shows which lectures are needed in order to solve task 1-7:

Task	Lectures
1	2, 3

Task	Lectures
2	4
3	3, 4
4	2, 4
5	3, 4
6	5
7	4

Task 8-10 can be solved after you have solved task 1-7.

Task 1:

Show that f and ι_S in (2) are convex and show that constraint qualification (CQ) holds. You are allowed to assume that $\operatorname{relint} S \neq \emptyset$. Note that f and ι_S also are closed, but you do not need to prove this.

Solution:

We note that f(x) is differentiable twice. The hessian becomes

$$\nabla^2 f(x) = Q$$

where $Q \in \mathbb{S}^n_{++}$ and thus convex. Let $(x, y) \in \mathbb{R}^n, \theta \in [0, 1]$. For a function to be convex, the inequality below needs to hold.

$$f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta) f(y)$$

Since we know that S convex, we know that all points on the line $\theta x + (1-\theta)y$ are inside the set per the definition meaning the function is convex.

Constraint qualification holds if f has full domain, which it has and g is proper. In order for a function to be proper, we require $\exists x$ such that $g(x) < \inf$, which is true when $x \in \mathbb{S}$

Task 2:

Compute the conjugate functions f^* and ι_S^*. The final expressions are not allowed to be given implicitly via optimization problems. E.g., projection formulas must be solved explicitly.

Solution:

$$\nabla f(x) = Qx + q$$

$$f^*(s) = \sup_{x \in \mathbb{R}^n} (s^T x - \frac{1}{2} x^T Q x - q^T x)$$

$$0 = s - \nabla f(x) \iff 0 = s - Qx - q \iff x = Q^{-1}(s - q)$$

$$f^*(s) = s^T Q^{-1}(s - q) - \frac{1}{2} Q^{-1} Q Q^{-1}(s - q) - q^T Q - 1(s - q)$$

$$f^*(s) = \frac{1}{2}(s - q)^T Q^{-1}(s - q)$$

We have that:

$$\iota_S^* = \sup_{x \in \mathbb{R}^n} (sx - \iota_S)$$

$$\iota_S^* = \sup_{x \in S} (sx)$$

We note that S is defined as:

$$S = \{x \in \mathbb{R}^n : a \leq x \leq b\}.$$

We remember that we are working with vector inequalities where $a_i \leq x_i \leq b_i$ for all $i = 1, \ldots, n$. This allows us to separate the $\sup$ for each i. Leading to:

$$\iota_S^* = \sum_{i=0}^{n} \sup_{a_i \leq x_i \leq b_i} (s_i x_i)$$

Finally, we note that the supremum of each of these will be at the endpoints, hence we can rewrite it as:

$$\iota_S^* = \sum_{i=0}^{n} \begin{cases} a_i s_i & s < 0 \\ b_i s_i & s > 0 \\ 0 & \text{else} \end{cases}$$

Task 3:

Write down a Fenchel dual problem to (2). Show that constraint qualification for the dual problem (CQ-D) holds.

Attention/hint: Keep track of your minus signs.

Solution:

The dual problem to (2) is:

$$\underset{\mu \in \mathbb{R}^n}{\text{minimize}} \; f^*(-\mu) + \iota_S^*(\mu).$$

CQ holds if f^* has full domain and g is proper. We see clearly that f has full domain as it is a quadratic function defined for all $\mu \in \mathbb{R}^n$. We can also clearly see that \iota^\ast_{S}(\mu) is proper meaning CQ-D holds.

Task 4:

Show that f and f^* are β-, and β^*-smooth, respectively. Find expressions for the smallest such parameters β and β^*. For instance, show that for any $\beta' < \beta$ we have that f is not β'-smooth.

Hint: Later when calculating the smoothness parameters in Python, make sure to read the documentation carefully so that you use the correct function.

Solution: A function is β-smooth if the following equations hold.

$$\|\nabla f(x) - \nabla f(y)\|_2 \leq \beta \|x - y\|_2 \tag{1}$$

For all $x, y \in \mathbb{R}^n$

$$\nabla f(x) = Qx + q \tag{2}$$

By inserting (2) in l.h.s of (1) we get:

$$\|Qx + q - (Qy + q)\|_2 = \|Q(x - y)\|_2$$

Recall that the spectral norm of $\|A\|_2$ of A is defined such that:

$$\|A\|_2 = \max\{\|Ax\|_2 : x \in \mathbb{R}^n, \|x\|_2 \leq 1\}$$

Which implies that:

$$\|Ax\|_2 \leq \|A\|_2 \|x\|_2$$

This gives:

$$\|Q(x - y)\|_2 \leq \|Q\|_2 \|(x - y)\|_2$$

Which shows that f is β-smooth for all $\|Q\|_2 \leq \beta$.

For f^* we have:

$$\nabla f^*(s) = Q^{-1}(s - q) \tag{3}$$

Inserting (3) in l.h.s of (1) we get:

$$\|Q^{-1}(x - q) - Q^{-1}(y - q)\|_2 = \|Q^{-1}(x - y)\|_2$$

Which, by using our knowledge of the spectral norm of a matrix gives us:

$$\|Q^{-1}(x - y)\|_2 \leq \|Q^{-1}\|_2 \|x - y\|_2$$

Which shows that $f^*(s)$ is β-smooth for all $\|Q^{-1}\|_2 \leq \beta$.

Task 5:

Compute ∇f, ∇f^*, $\text{prox}_{\gamma\iota_S}$ and $\text{prox}_{\gamma\iota_S^*}$. The final expressions are not allowed to be given implicitly via optimization problems. E.g., projection formulas must be solved explicitly.

Solution: From task 4, we have the following:

$$\nabla f(x) = Qx + q$$

$$\nabla f^*(s) = Q^{-1}(s - q)$$

Computing $\text{prox}_{\gamma\iota_S}$ is done by the following:

For the function ι_S we have:

$$g(x) = \begin{cases} 0 & \text{if } a \leq x \leq b \\ \infty & else \end{cases}$$

Which gives the subgradient:

$$\partial g(x) = \begin{cases} (-\infty, 0] & \text{if } x = a \\ \{0\} & \text{if } a \leq x \leq b \\ [0, \infty) & \text{if } x = b \\ \emptyset & \text{else} \end{cases}$$

Using Fermat's rule we get $\text{prox}_{\gamma g}(z) : 0 \in \partial g(x) + \gamma^{-1}(x - z)$ This gives for the different cases:

$$\text{prox}_{\gamma\iota_S^*}(z) = \begin{cases} a & \text{if } s \leq a \\ s & \text{if } a < s < b \\ b & \text{if } s \geq b \end{cases}$$

For the function ι_S^* we have:

$$g^*(s) = \sum_{i=0}^{n} \begin{cases} a_i s_i & s_i < 0 \\ b_i s_i & s_i > 0 \\ 0 & \text{else} \end{cases}$$

We note that g(s) is a separable function, giving us subdifferential for each $g_i(s)$ This gives us the following subdifferentials:

$$\partial g_i^*(s_i) = \begin{cases} a & \text{if } s_i \leq a \\ [a, b] & \text{if } a < s_i < b \\ b & \text{if } s_i \geq b \end{cases}$$

Using Fermat's rule we get $\text{prox}_{\gamma g}(z) : 0 \in \partial g(x) + \gamma^{-1}(x - z)$ This gives for the different cases:

$$\text{prox}_{\gamma\iota_S^*}(z) = \begin{cases} z - \gamma a & \text{if } z \leq \gamma a \\ 0 & \text{if } \gamma a < z < \gamma b \\ z - \gamma b & \text{if } z \geq \gamma b \end{cases}$$

Task 6:

Based on your results above, write explicitly out the proximal gradient update rule (4) for both the primal and the dual problem. Use x as the primal variable and μ as the dual variable.

Attention/hint: Keep track of your minus signs.

Solution: By using (4) we have that:

$$x^{k+1} = \text{prox}_{\gamma g}\big(x^k - \gamma(Qx^k + q)\big)$$

$$\mu^{k+1} = \text{prox}_{\gamma g^*}\big(\mu^k + \gamma(Q^{-1}(-\mu^k - q))\big)$$

Inserting our values from Task 5 gives us:

$$x^{k+1} = \begin{cases} a & \text{if } \big(x^k - \gamma(Qx^k + q)\big) \leq a \\ \big(x^k - \gamma(Qx^k + q)\big) & \text{if } a < \big(x^k - \gamma(Qx^k + q)\big) < b \\ b & \text{if } \big(x^k - \gamma(Qx^k + q)\big) \geq b \end{cases}$$

$$\mu^{k+1} = \begin{cases} \big(\mu^k + \gamma(Q^{-1}(-\mu^k - q))\big) - \gamma a & \text{if } \big(\mu^k + \gamma(Q^{-1}(-\mu^k - q))\big) \leq \gamma a \\ 0 & \text{if } \gamma a < \big(\mu^k - \gamma(Q^{-1}(\mu^k + q))\big) < \gamma b \\ \big(\mu^k + \gamma(Q^{-1}(-\mu^k - q))\big) - \gamma b & \text{if } \big(\mu^k + \gamma(Q^{-1}(-\mu^k - q))\big) \geq \gamma b \end{cases}$$

Task 7:

Suppose that $\mu^\star \in \mathbb{R}^n$ is an optimal solution to the dual problem you found in Task 3. Given $\mu^\star$, and **starting from the optimality condition for the dual problem (given by *Fermat's rule*)**, recover an optimal point $x^\star \in \mathbb{R}^n$ to the primal problem (2), and show that this $x^\star$ is in fact an optimal solution to the primal problem (2). Motivate each step in your argument carefully.

Solution:

Assuming we have $\mu^\star \in \mathbb{R}^n$, then the following must hold:

$$0 \in \partial f^*(\mu) - L\partial g^*(-L^T\mu)$$

Then the optimal primal x must satisfy any of the primal-dual problems:

$$\begin{cases} \mu \in \partial f(Lx) \\ -L^T\mu \in \partial g(x) \end{cases} \qquad \begin{cases} Lx \in \partial f^*(\mu) \\ -L^T\mu \in \partial g(x) \end{cases}$$

$$\begin{cases} \mu \in \partial f(Lx) \\ x \in \partial g^*(-L^T\mu) \end{cases} \qquad \begin{cases} Lx \in \partial f^*(\mu) \\ x \in \partial g^*(-L^T\mu) \end{cases}$$

We see that for our case, since $\partial f^*(\mu)$ is differentiable at all points $Lx = \partial f^*(\mu)$ is a solution to our problem where I is the identity matrix.

Task 8:

Use your results above to fill in the functions below.

Solution:

In [68]:
```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

def quad(x,Q,q):
    """
    quad(x,Q,q) computes the quadratic function (1/2)x'Qx + q'x

    :param x: the variable of the quadratic function
    :param Q: the matrix in the quadratic function that corresponds to th
    :param q: the vector in the quadratic function that corresponds to th
    :return: (1/2)x'Qx + q'x
    """
    res = (1 / 2) * x.T @ Q @ x + q.T @ x
    return res

def quadconj(mu,Q,q):
    """
    quadconj(mu,Q,q) computes the conjugate function of the
    quadratic function (1/2)x'Qx + q'x, evaluated at mu

    :param mu: the variable of the conjugate function
    :param Q: the matrix in the quadratic function that corresponds to th
    :param q: the vector in the quadratic function that corresponds to th
    :return: conjugate of (1/2)x'Qx + q'x, evaluated at mu
    """
    conj = (1/2) * (mu - q).T @ np.linalg.inv(Q) @ (mu - q)
    return conj

def box(x,a,b):
    """
    box(x,a,b) computes the indicator function of the box contraint
    [a,b]

    :param x: the variable of the indicator function
    :param a: the left vector defining the box contraint
    :param b: the right vector defining the box contraint
    :return: 0 if x is in [a,b] and infinity otherwise
    """
    if np.all(a <= x) and np.all(x <= b):
        return 0
    else:
        return np.inf

def boxconj(mu,a,b):
    """
    boxconj(mu,a,b) computes the conjugate function of the indicator func
    of the box contraint [a,b], evaluated at mu

    :param mu: the variable of the conjugate function
    :param a: the left vector defining the box contraint
    :param b: the right vector defining the box contraint
```

```
    :return: conjugate of the indicator function of the box contraint [a,
    """
    return np.sum(np.maximum(a * mu, b * mu))



def grad_quad(x,Q,q):
    """
    grad_quad(x,Q,q) computes the gradient of the quadratic function (1/2

    :param x: the variable of the quadratic function
    :param Q: the matrix in the quadratic function that corresponds to th
    :param q: the vector in the quadratic function that corresponds to th
    :return: gradient of (1/2)x'Qx + q'x
    """
    grad = Q @ x + q
    return grad

def grad_quadconj(mu,Q,q):
    """
    grad_quadconj(mu,Q,q) computes the gradient of the conjugate function
    the quadratic function (1/2)x'Qx + q'x, evaluated at mu

    :param mu: the variable of the conjugate function
    :param Q: the matrix in the quadratic function that corresponds to th
    :param q: the vector in the quadratic function that corresponds to th
    :return: gradient of the conjugate of (1/2)x'Qx + q'x, evaluated at m
    """
    return np.linalg.inv(Q) @ (mu - q)

def prox_box(x,a,b,gamma):
    """
    prox_box(x,a,b,gamma) computes proximal operator of the indicator fun
    of the box contraint [a,b], evaluated at x

    :param x: the variable of the poximal operator
    :param a: the left vector defining the box contraint
    :param b: the right vector defining the box contraint
    :param gamma: the step-size parameter
    :return: proximal operator of the indicator function of the
    box contraint [a,b], evaluated at x
    """
    v = x.copy()
    for i in range(x.size):
        if x[i] <= a[i]:
            v[i] = a[i]
        elif x[i] >= b[i]:
            v[i] = b[i]
        else:
            v[i] = x[i]
    return v

def prox_boxconj(mu,a,b,gamma):
    """
    prox_box(mu,a,b,gamma) computes proximal operator of the conjugate fu
    the indicator function of the box contraint [a,b], evaluated at mu

    :param mu: the variable of the poximal operator
    :param a: the left vector defining the box contraint
    :param b: the right vector defining the box contraint
```

```
    :param gamma: the step-size parameter
    :return: proximal operator of the conjugate function of the indicator
    box contraint [a,b], evaluated at mu
    """
    v = mu.copy()
    for i in range(mu.size):
        if mu[i]  <= gamma * a[i] :
            v[i] = mu[i] - gamma * a[i]
        elif mu[i]  >= gamma *b[i]:
            v[i] = mu[i] - gamma * b[i]
        else:
            v[i] = 0
    return v

def dual_to_primal(mu,Q,q,a,b):
    """
    dual_to_primal(mu,Q,q,a,b) computes the solution x* to the primal pro
    given a solution mu* to the dual problem.

    :param mu: the dual variable
    :param Q: the matrix in the quadratic function that corresponds to th
    :param q: the vector in the quadratic function that corresponds to th
    :param a: the left vector defining the box contraint
    :param b: the right vector defining the box contraint
    :return: the extracted primal variable
    """
    return np.linalg.inv(Q) @ - (mu + q)
```

Task 9:

Below is a function for generating Q, q, a, and b that define the quadratic function f and the box constraint set S. Use Task 8 to solve the primal problem using the proximal gradient method.

In [69]:
```
def problem_data():
    """
    problem_data() generates the problem data variables Q, q, a and b

    :return: (Q,q,a,b)
    """
    rs = np.random.RandomState(np.random.MT19937(np.random.SeedSequence(1
    n = 20
    Q = rs.randn(n,n)
    Q = Q.T@Q
    q = rs.randn(n)
    a = -rs.rand(n)
    b = rs.rand(n)
    return Q, q, a, b

(Q,q,a,b) = problem_data()

def prox_grad(gamma, x, Q, q, a, b, tol=1e-7, max_iter=10000):
    res = []
    for k in range(max_iter):
        x_next = prox_box(x - gamma * grad_quad(x, Q, q), a, b, gamma)
```

```
        r = np.linalg.norm(x_next - x)
        res.append(r)
        x = x_next
        if r < tol:
            break
    return x, k, res
```

a) What seems to be the best choice of γ?

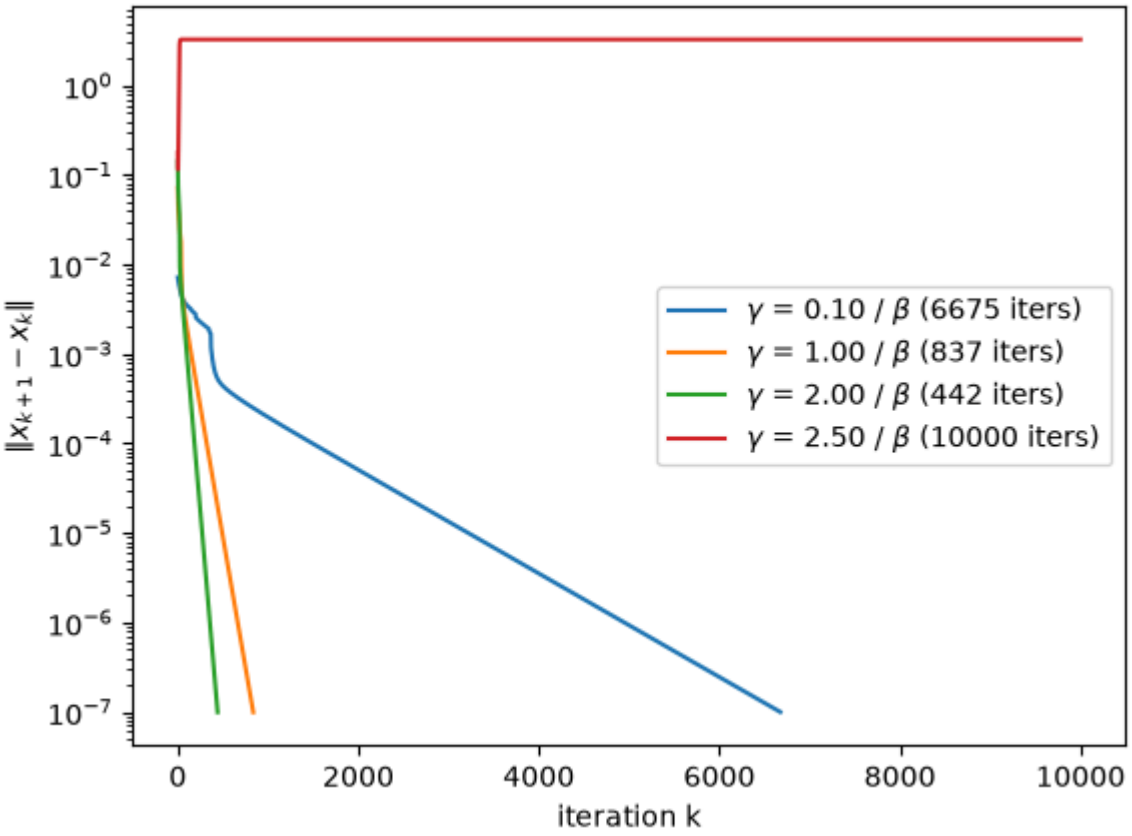
Solution:

Fill in your solution here!

In [112…

```
def plot_residuals(gammas, x_init, Q, q, a, b):
    fig, ax = plt.subplots()
    for gamma in gammas:
        _, n_iter, res = prox_grad(gamma[1], x_init, Q, q, a, b)
        ax.semilogy(res, label=f"{r"$\gamma$"} = {gamma[0]:.2f} / {r"$\be
    ax.set_xlabel("iteration k")
    ax.set_ylabel(r"$\|x_{k+1} - x_k\|$")
    ax.legend()
    plt.show()

beta= np.linalg.eigvalsh(Q).max()
gammas = [(c, c / beta) for c in (0.1, 1.0, 2, 2.5)]
plot_residuals(gammas, np.zeros_like(q), Q, q, a, b)
```



b) Does the upper bound $\gamma < \frac{2}{\beta}$ seem reasonable?

Solution:

Yes, the plot clearly shows that larger values for gamma does not ensure convergence.

Test different initial points for the algorithm:

c) Does this affect the point the algorithm converges to?

Solution:

No. The code shows the distance between the different points the algorithm converges to based on a random starting point is negligible.

In [ ]:
```
x_final_res = []
for _ in range(10):
  (Q,q,a,b) = problem_data()
  x = np.random.randn(q.shape[0])
  x_final, iter, res_norm_result = prox_grad(1/np.linalg.eigvalsh(Q).max(
  x_final_res.append(x_final)


diff_res = []
for i in range(1, len(x_final_res)):
  x_ref = x_final_res[0]
  diff = np.linalg.norm(x_ref - x_final_res[i])
  diff_res.append((i, diff))

for res in diff_res:
  print(f"x_final: {res[0]} \tDistance: {res[1].item()}")
```

```
X_final: 1      Distance: 4.700314811358064e-08
X_final: 2      Distance: 2.9142662529461776e-08
X_final: 3      Distance: 2.4957272951304914e-08
X_final: 4      Distance: 1.4725631366430568e-05
X_final: 5      Distance: 1.2347638849323296e-08
X_final: 6      Distance: 2.8532241600542668e-08
X_final: 7      Distance: 2.7490944366425516e-08
X_final: 8      Distance: 3.1520555799880226e-08
X_final: 9      Distance: 6.520857532167805e-08
```

d) Carefully motivate theoretically why/why not it affects the final point. *Hint:* Look at the objective function in (2).

Solution:

Since $\nabla^2 f(x) = Q$ is positive definite, it is strong convex. We have proven that $g(x)$ is convex and adding a convex function to a strong convex function keeps the strong property. A strong convex function has a unique minimizing point, making a local minimum the global minimum.

e) Does your final point x^{final} satisfy the constraint $x^{\text{final}} \in S$?

Solution:

Yes. If not, x^{final} would be infinity, as all points outside S are infinity by the indicator function.

In [72]:
```
(Q,q,a,b) = problem_data()
x = np.random.randn(q.shape[0])
x_final, iter, res_norm_result = prox_grad(1/np.linalg.eigvalsh(Q).max(),

x_final == x_final + box(x_final, a, b)
```

Out[72]:
```
array([ True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True])
```

f) What about the iterates, do they always satisfy the constraint, $x^k \in S$? Why/why not?

Solution:

Since $f(x)$ is convex, the prox operator will always move us closer to x^{final}. Since x^{final} is inside S, a point on the way will never be outside of S.

Task 10:

Solve the dual problem.

a) Similar to the previous task, find/verify the upper bound on the step-size and find a good step-size choice.

Solution:

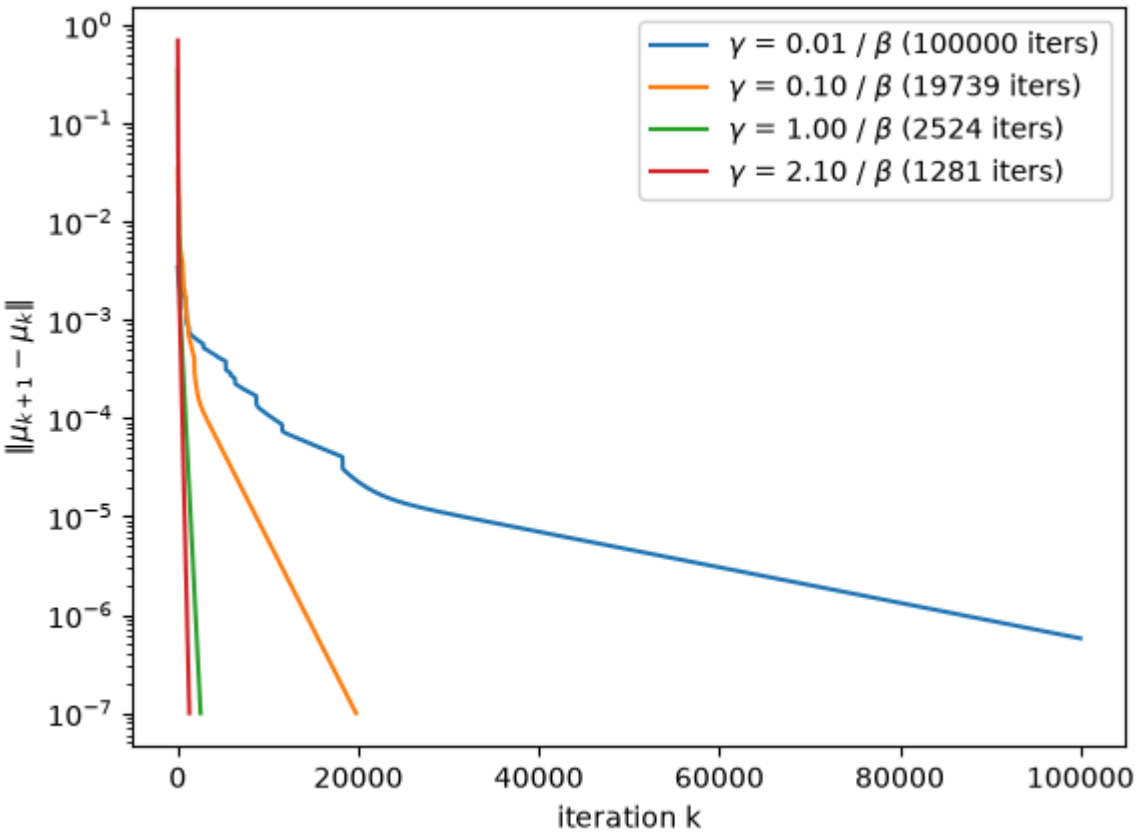
Fill in your solution here!

In [136…
```
def prox_grad_dual(gamma, mu, Q, q, a, b, tol=1e-7, max_iter=100000):
    res = []
    mu_k = []
    for k in range(max_iter):
        mu_next = prox_boxconj(mu + gamma * grad_quadconj(-mu, Q, q), a,
        r = np.linalg.norm(mu_next - mu)
        res.append(r)
        mu = mu_next
        mu_k.append(mu)
        if r < tol:
            break
    return mu, mu_k, k, res

def plot_residuals(gammas, x_init, Q, q, a, b):
    fig, ax = plt.subplots()
    for gamma in gammas:
        mu, _, n_iter, res = prox_grad_dual(gamma[1], x_init, Q, q, a, b)
        ax.semilogy(res, label=f"{r"$\gamma$"} = {gamma[0]:.2f} / {r"$\be
    ax.set_xlabel("iteration k")
    ax.set_ylabel(r"$\|\mu_{k+1} - \mu_k\|$")
    ax.legend()
    plt.show()

beta= 1 / np.linalg.eigvalsh(Q).min()
```

```
gammas = [(c, c / beta) for c in (0.01, 0.1, 1, 2.1)]
plot_residuals(gammas, np.random.randn(q.shape[0]), Q, q, a, b)
```



Let x^{final} be the final points from Task 9 and μ^{final} the final point for the dual problem. Let $\hat{x}^{\text{final}}$ final primal points extracted from the final dual point μ^{final} using the expression from Task 7:

b) Are x^{final} and $\hat{x}^{\text{final}}$ the same?

Solution:

Fill in your solution here! We see that the points x^{final} and $\hat{x}^{\text{final}}$ are not identical but due to rounding issues they are identical to the 4th decimal.

In [137…

```
(Q,q,a,b) = problem_data()
mu_init = np.random.randn(q.shape[0])
x_init = mu_init.copy()
x_final, iter_x, res_norm_result_x = prox_grad(1/np.linalg.eigvalsh(Q).ma
mu_final, mu_k, iter, res_norm_result = prox_grad_dual(np.linalg.eigvalsh

x_final_from_dual = dual_to_primal(mu_final, Q, q, a, b)
eq_precision = []
for i in range(1,11):
  eq_precision.append((i, np.all(np.round(x_final_from_dual, i) == np.rou
for res in eq_precision:
  print(f"Decimals: {res[0]} \tEqual points: {res[1].item()}")
```

```
Decimals: 1     Equal points: True
Decimals: 2     Equal points: True
Decimals: 3     Equal points: True
Decimals: 4     Equal points: True
Decimals: 5     Equal points: False
Decimals: 6     Equal points: False
Decimals: 7     Equal points: False
Decimals: 8     Equal points: False
Decimals: 9     Equal points: False
Decimals: 10    Equal points: False
```

c) Is $\hat{x}^{\text{final}}$ in the box S?

Solution:

No, the code below shows that adding the indicator function to the result does not produce an identical result as the function itself.

In [139…
```
inside_box = []
for i in range(1,11):
  inside_box.append((i, np.all(np.round(x_final_from_dual, i) == np.clip(
for res in inside_box:
  print(f"Decimals: {res[0]} \tInside box: {res[1].item()}")
```

```
Decimals: 1     Inside box: False
Decimals: 2     Inside box: False
Decimals: 3     Inside box: False
Decimals: 4     Inside box: False
Decimals: 5     Inside box: False
Decimals: 6     Inside box: False
Decimals: 7     Inside box: False
Decimals: 8     Inside box: False
Decimals: 9     Inside box: False
Decimals: 10    Inside box: False
```

d) Let μ^k be the iterates of the dual method, using the expression from Task 7, extract the primal iterates $\hat{x}^k$ from μ^k. Does $\hat{x}^k$ always satisfy the constraint $\hat{x}^k \in S$?

Solution:

No, the code below shows that there exists a value $\hat{x}^k$ that does not satisfy the constraint.

In [140…
```
(Q,q,a,b) = problem_data()
mu_init = np.random.randn(q.shape[0])
_, mu_k, _, _ = prox_grad_dual(np.linalg.eigvalsh(Q).min(), mu_init, Q, q

for mu in mu_k:
  x_k = dual_to_primal(mu, Q, q, a, b)
  if np.any(x_k != x_k + box(x_k, a, b)):
    print(r"x_k not in S")
    break
```

```
x_k not in S
```

e) How do the function values $f\left(\hat{x}^k\right)$ develop over the iterations?

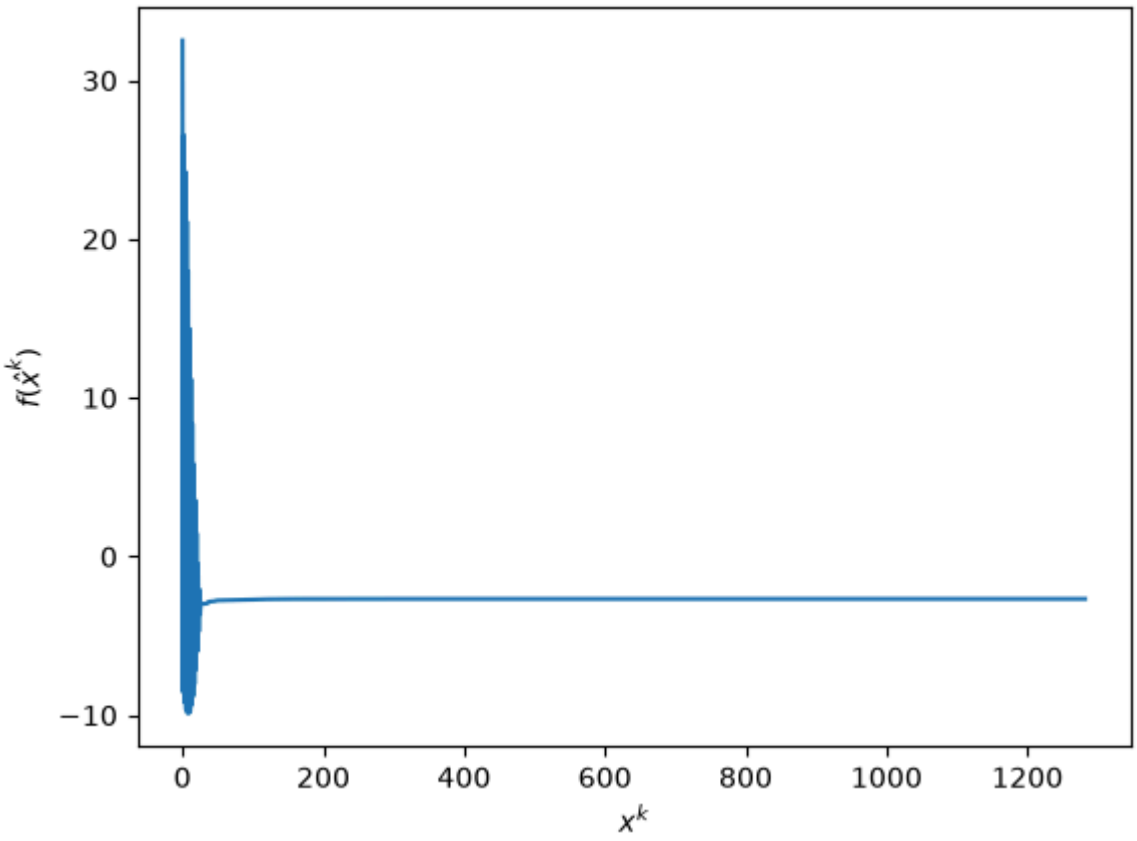
Solution:

In [142…

```
(Q,q,a,b) = problem_data()
mu_init = np.random.randn(q.shape[0])
_, mu_k, _, _ = prox_grad_dual(2 * np.linalg.eigvalsh(Q).min(), mu_init,

f_x_k = []
for mu in mu_k:
  x_k = dual_to_primal(mu, Q, q, a, b)
  f_x_k.append(quad(x_k, Q, q))

plt.xlabel(r"$x^k$")
plt.ylabel(r"$f(\hat{x}^k)$")
plt.plot(f_x_k)
plt.show()
```



f) What about $f\left(\hat{x}^k\right) + \iota_S\left(\hat{x}^k\right)$?

Solution:

Since we can't ensure the prox of the conjugate clips us in to the box, we can't deduce anything of the value of $f\left(\hat{x}^k\right) + \iota_S\left(\hat{x}^k\right)$. It is either $f\left(\hat{x}^k\right)$ if we happen to be in the box, or $f\left(\hat{x}^k\right) + \iota_S\left(\hat{x}^k\right) = \infty$ if we are outside the box. In theory, if we took infinitely many iterations, we would reach $f(x^\star)$